

CS1632: Systems Testing the Web with Playwright

Wonsun Ahn

Background

- Systems testing: testing the entire system as a whole
 - We would like to automate systems testing, just like we did for unit testing
- So far, all of our testing has been text-based java programs
 - A.k.a. Command Line Interface (CLI) programs
- Automating testing for CLI programs is easy!
 - Just create an “input script” and redirect to stdin
 - Redirect stdout to file and compare to expected output

Automated Systems Test for CLI Programs Using Bash

```
#!/bin/bash
```

```
# Test case 1: Rent out cat #2 and check cat list
```

```
echo -e "2\n2\n1\n5\n" | java -jar rentacat.jar >  
testcase1.observed.out
```

```
diff testcase1.observed.out testcase1.expected.out
```

```
...
```

Automated Systems Test for GUI Programs

- Turns out that not every program is a CLI program
 - Web pages, mobile applications, windows applications, etc.
 - A.k.a. Graphical User Interface (GUI) programs
 - How do we deal with these?
- The theory behind testing remain the same
 - Compare **observed behavior vs. expected behavior**
 - Preconditions
 - Execution steps
 - Postconditions
- But we need different tools to automate testing GUI programs

Insight: GUI Apps $\approx$ Text-based Apps

- GUI apps also have a text representation for the output
 - It's just that the text is rendered into a graphical representation for end-user
- Example: Web applications with HTML (HyperText Markup Language)
 - HTML text is fetched from web server when a URL is requested
 - HTML text is rendered by web browser into graphical elements
- Example: Mobile applications with XML (Extensible Markup Language)
 - XML text is fetched from mobile app server or generated by app
 - XML text is rendered by Android / iOS into graphical elements
- So, in theory, GUI apps could be tested just like text-based apps
 - Using a simple string comparison of expected and observed output text
 - Assuming rendering is bug-free (safe to assume for modern browsers, Android / iOS)
 - But modern web apps build much of the page with JavaScript at runtime
 - So the fetched HTML is often not the page the user actually sees

Testing a Web App like a Text-based App

```
#!/bin/bash
```

```
# Test case 1: Fetch index.html and compare
```

```
wget example.com/index.html
```

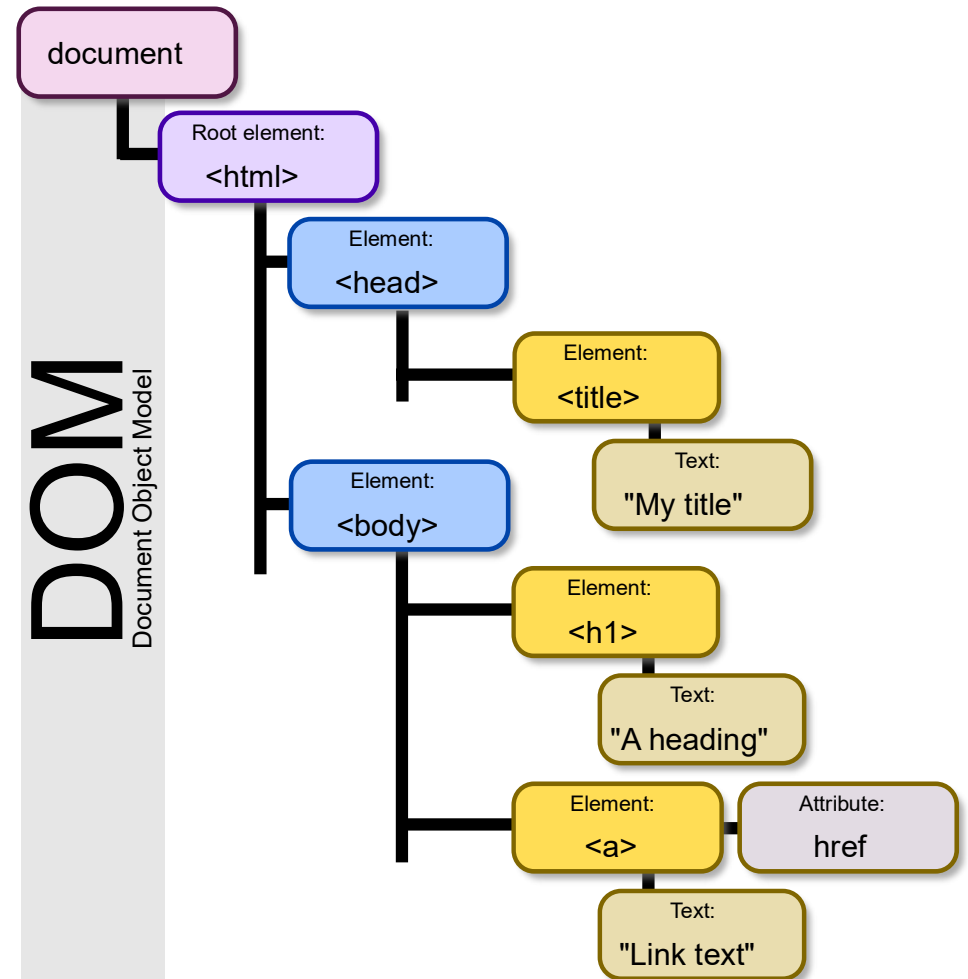
```
diff index.html index.expected.html
```

```
...
```

1. HTML page is fetched from web server using `wget` utility
2. Fetched HTML page is compared against the expected HTML page
 - Do you see any problems with this testing methodology?

HTML is organized into a hierarchy of elements

- Element: tag + content + attributes
- Tag: specifies type of element
 - <title>, <table>, <button>, , ...
 - <script>: JavaScript code that runs on browser
- Content: text to be displayed
- Attribute:
 - Specifies layout of element
 - Specifies action performed on interaction
- This model of web page is called DOM
 - DOM: Document Object Model



Problems with Naïve HTML Comparison

1. Tests are fragile
 - Trivial changes in HTML that don't impact final display can break test
2. Tests are untargeted
 - Changing HTML elements unrelated to test target will break test
3. JavaScript code is not functionally tested
 - JS code is compared verbatim to expected JS code, instead of executed
 - Changes in JS code that don't change functionality (e.g. commenting) breaks test
4. The fetched HTML is not the rendered page
 - Modern apps build much of the DOM with JavaScript after the page loads
 - wget sees the initial shell, not what the user actually sees

1. Tests are Fragile

- Are the two really different? (Hint: No, HTML tag names are case-insensitive)

[Expected HTML]

```
<html>
<head>
  <title>Example</title>
</head>
<body>
  ...
</body>
</html>
```

[Observed HTML]

```
<HTML>
<HEAD>
  <TITLE>Example</TITLE>
</HEAD>
<BODY>
  ...
</BODY>
</HTML>
```

2. Tests are Untargeted

- Which HTML element are we testing?

```
<html>
<head>
  <title>Example Domain</title>
  <style type="text/css">
    a:link, a:visited {
      color: #38488f;
    }
  </style>
</head>
<body>
  <a href="https://www.iana.org/domains/example">More info</a>
</body>
</html>
```

Diagram illustrating the HTML structure and associated test targets:

- Title text?** points to the `<title>Example Domain</title>` element.
- CSS URL link color?** points to the `color: #38488f;` CSS rule.
- URL link?** points to the `https://www.iana.org/domains/example` href attribute.
- Body text?** points to the `More info` text content of the link.

3. JavaScript code is not functionally tested

- Are the two really different when displayed? (Hint: No)

[Expected HTML]

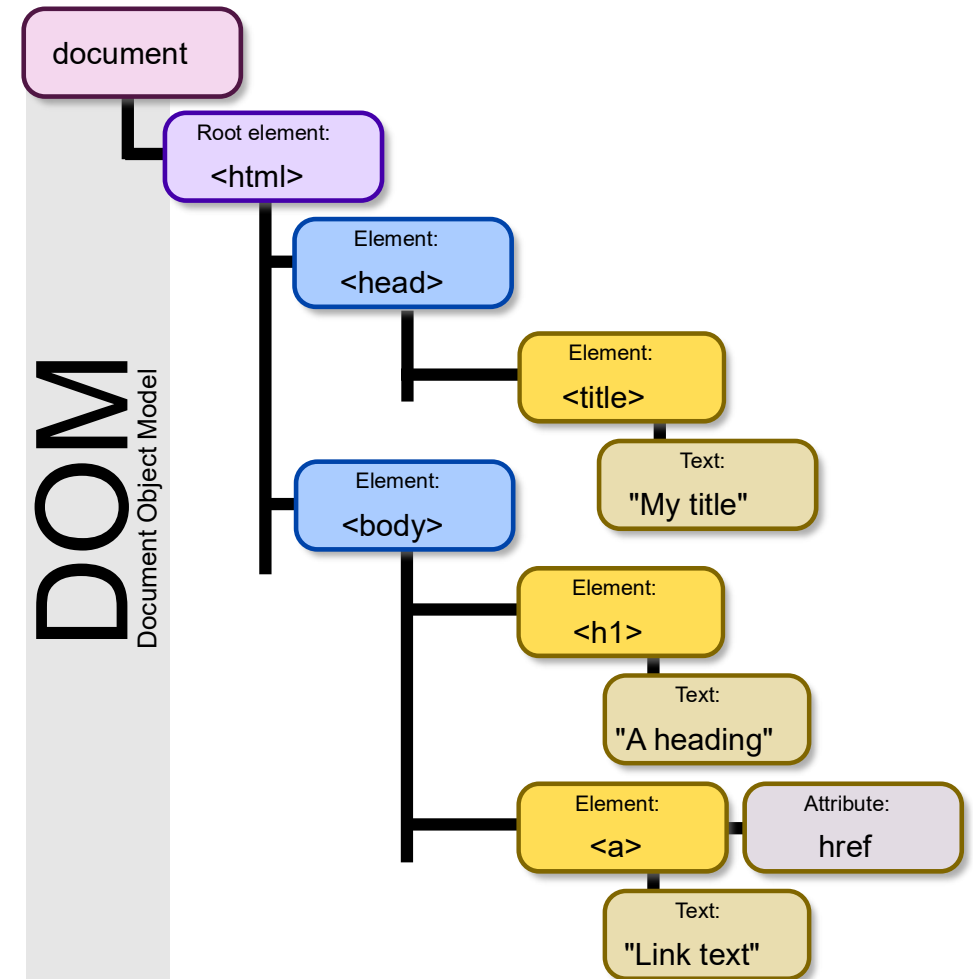
```
<html>
  <head>
    <script>
      function doAlert() {
        var msg = 'hello';
        alert(msg);
      }
    </script>
  </head>
  <body onload="doAlert();"></body>
</html>
```

[Observed HTML]

```
<html>
  <head>
    <script>
      function doAlert() {
        var message = 'hello';
        alert(message);
      }
    </script>
  </head>
  <body onload="doAlert();"></body>
</html>
```

Solution: Web Testing Frameworks

- **Web testing framework:** Framework for testing web apps at a semantic level
 - **Robust:** Parses HTML into a DOM tree, so HTML formatting changes don't break tests
 - **Targeted:** Uses locators to target individual HTML elements for interacting and testing
 - **Functionally tested:**
 - Provides APIs to call JavaScript code
 - Can emulate events like clicking or typing, which in turn invokes JavaScript code
- Examples: Selenium, **Playwright**, Cypress



Playwright works on Accessibility Trees

- Locators query the **accessibility tree** the browser builds from the DOM, not the DOM itself

[DOM tree: XPath/CSS locators in Selenium]

```
<div class="hdr">  
  <a href="/node/606">  
    <img alt="University of Pittsburgh">
```

XPath: `//div[@class='hdr']/a/img`

Selects by: tag, attribute, position of element in DOM

[Accessibility tree: Playwright]

```
banner "Site header"  
  link "University of Pittsburgh"  
    img "University of Pittsburgh"
```

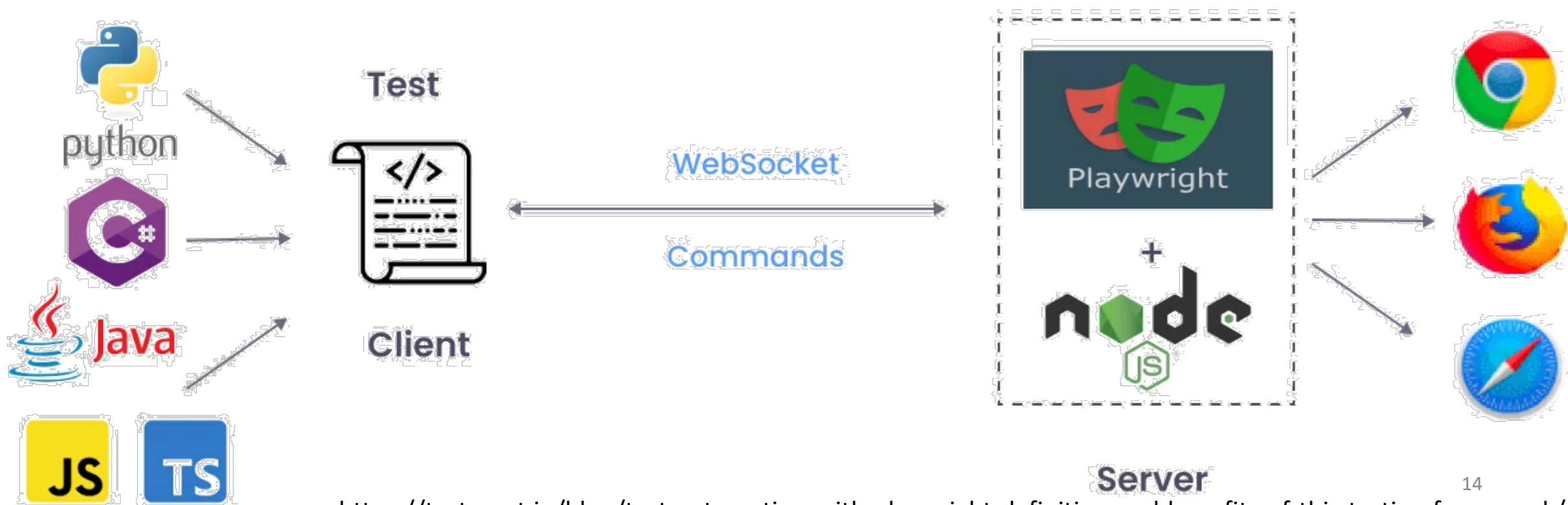
Playwright: `getByRole('link',
 { name: 'University of Pittsburgh' })`

Selects by: role, name and state of element accessible to user

- Structural changes to the DOM tree that do not change how users access the site (e.g. Renaming the class, adding a `<div>`, swapping `<div>` for `<header>`)
→ Can break Selenium XPath/CSS locators but does not break Playwright built-in locators
- Playwright locators break when the site's user-facing semantics change, not when HTML is restructured.**

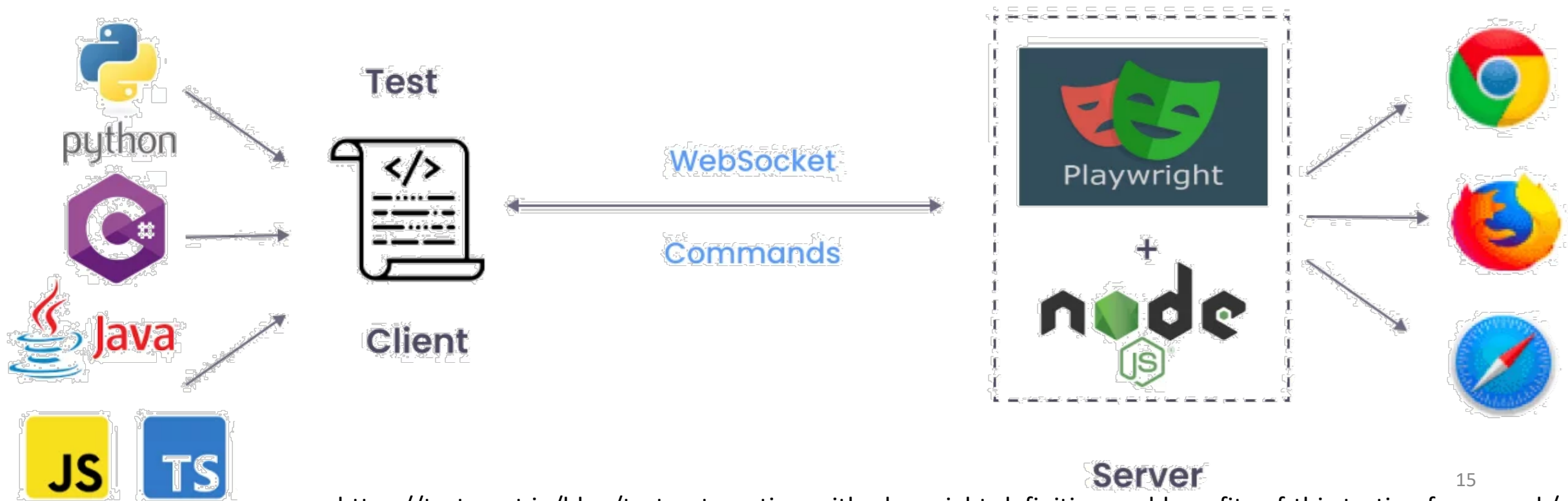
Playwright is cross-language / cross-browser

- Node.js server maintains websocket connections with client and browsers
 1. Test client sends JSON commands to Node.js server
 2. Node.js server translates commands to native protocol messages for browsers



Playwright works **asynchronously**

- It takes an indefinite amount of time between command and response
 - Calling a test API returns a **promise** (a placeholder for a future result)
 - Test client must be careful of running ahead of browser and causing race conditions



`await` orders Playwright calls

- Consider the following test code:

```
test('TEST-2-LOGO-EXISTS', async ({ page }) => {  
  // Loads the URL https://www.pitt.edu  
  await page.goto('https://www.pitt.edu/');  
  // Verifies a link with the name 'University of Pittsburgh' is visible  
  await expect(page.getByRole('link', { name: 'University of  
Pittsburgh', exact: true })).toBeVisible();  
});
```

- Calling `await` on the promise returned by each asynchronous Playwright call orders the actions.
- Without it, the script may run ahead and terminate before the postcondition is verified.

Retry mechanism does implicit synchronization

- `await` orders your calls. The retry loop inside `expect()` waits for the page.

[Snapshot: no retry]

```
const items = page.getByRole('listitem');  
expect(await items.count()).toBe(16);
```

count() runs once, never re-checks

Fails when the list has not rendered yet

[Retrying assertion: waits]

```
const items = page.getByRole('listitem');  
await expect(items).toHaveLength(16);
```

re-counts until 16, up to 5 s

Passes as soon as the list renders

- `expect()` will only retry an assertion if it is passed a locator or page object.
- **Give `expect()` a locator it can re-evaluate, not a value you already computed.**

Resources

- Playwright best practices guide:
<https://playwright.dev/docs/best-practices>
- Playwright locators: <https://playwright.dev/docs/locators>
- Playwright mechanisms that prevent race conditions
 - JavaScript async/await mechanism:
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/await>
 - Playwright auto-waiting:
<https://playwright.dev/docs/actionability>
- Try sample_code/playwright_example to get some practice